b) RM est-il optimal pour S3 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de chaque tâche, et conclure.
e) La simulation est-elle exacte, pessimiste ou optimiste pour S3 ?
f) Tracer une simulation et comparer les pires temps de réponse observés par rapport à ceux obtenus par la RTA. Comment expliquer les différences observées ?

Exercice 5

Soit le système S4 de deux tâches indépendantes, sporadiques, dont les paramètres < C_i, D_i, T_i> sont donnés par {<26,26,70>, <62,118,100>}, ordonnancé suivant DM. Cet exemple est tiré de (Lehoczky, 1990) et a permis d'illustrer le fait que ce n'est pas forcément la première instance qui possède le plus long temps de réponse, mais l'une des instances se trouvant dans la plus longue période d'activité.

a) Calculer la charge du système S3, et conclure.
b) DM est-il optimal pour S4 ?
c) La RTA est-elle exacte, pessimiste ou optimiste ?
d) Calculer avec la RTA le temps de réponse de la première instance de τ_2.
e) Calculer avec la RTA le temps de réponse de la seconde instance de τ_2.
f) En réalité, on trouve 7 instances de la tâche τ_2 dans la période d'activité initiée par l'instant critique, et c'est la 5ème instance qui possède le plus long temps de réponse. Calculer ce temps de réponse.

Exercice 6

En observant la RBF donnée sur la Figure 3, qu'adviendrait-il de la durée de la période d'activité si :

a) Une durée d'exécution de tâche était moins longue ?
b) Une période était plus longue que prévue ?
c) Une date de réveil était plus grande que 0 ?
d) Peut-on en déduire que la RTA est C-viable, r-viable, et T-viable ?

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Eisenbrand, F., & Rothvos, T. (2008). Static-priority real-time scheduling: Response time computation is NP-hard. *Real-Time Systems Symposium*, (pp. 397-406).

Joseph, M., & Pandya, P. (1986). Finding response times in a real-time system. *The Computer Journal 29 (5)*, 390-395.

Lehoczky, J. P. (1990). Fixed priority scheduling of periodic task sets with arbitrary deadlines. *Proc. of the 11th Real-Time Systems Symposium*, (pp. 201-209).

TD VAT 5
Analyse de temps de réponse de tâches dépendantes

Exclusions mutuelles

Nous avons évoqué le problème d'**inversion de priorité** : sans modification de la priorité de tâches partageant des ressources, tout se passe comme si une tâche prioritaire attendant une ressource détenue par une tâche moins prioritaire se voyait hériter de la priorité de cette dernière. Ainsi, des tâches de priorité intermédiaires s'exécuteraient avant la tâche prioritaire bloquée. Ceci est illustré sur la Figure 1.

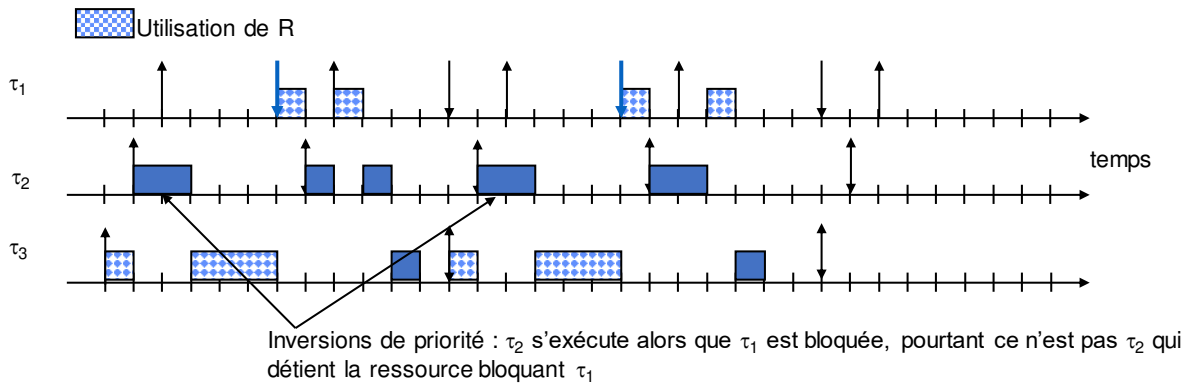


Figure 1 : Inversion de priorité

Deux protocoles de gestion de ressources sont proposés dans (Sha, Rajkumar, & Lehoczky, 1990), le **protocole à priorité héritée** (PIP – Priority Inheritance Protocol), qui consiste, lorsqu'une tâche cherche à prendre un mutex qui est déjà détenu par une autre tâche, à faire temporairement hériter de sa priorité la tâche détenant la ressource. Ce protocole est illustré sur la Figure 2.

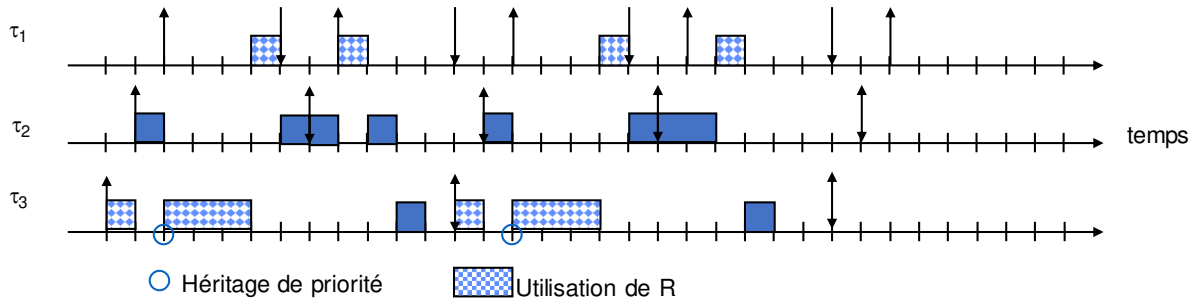


Figure 2 : Protocole à priorité héritée

Dans ce cas, on évite l'inversion de priorité, cependant :

- Il y a possibilité d'interblocage (voir problème des philosophes) ;
- Une tâche devant prendre plusieurs mutex peut se trouver bloquer par une section critique moins prioritaire pour chaque mutex (voir Figure 3).

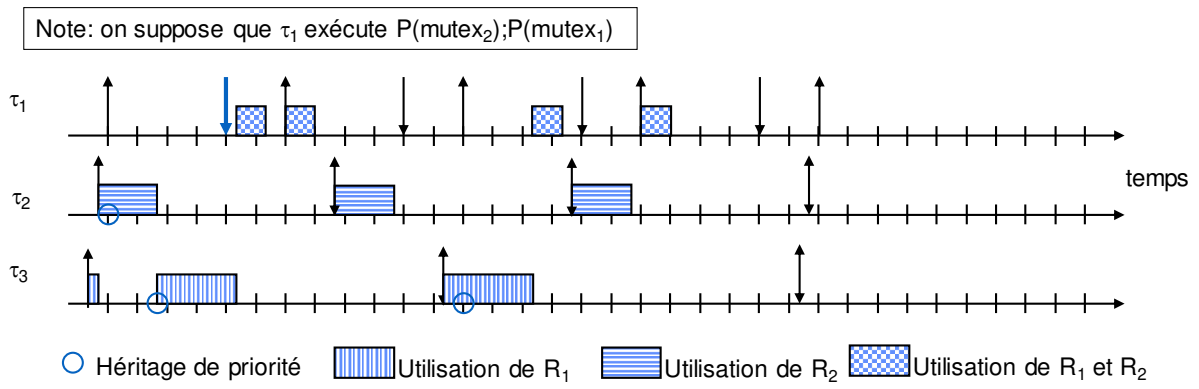


Figure 3 : Avec le protocole à priorité héritée, chaque demande de mutex peut donner lieu à attente de fin de section critique

C'est pour cela que ce même article propose un **protocole à priorité plafond** (PCP - Priority Ceiling Protocol) : chaque mutex se voit doter d'une priorité dite plafond, égale à la priorité maximale des tâches susceptibles de prendre le mutex. Le système se voit muni d'une valeur de plafond, égale à la priorité plafond maximale des mutex en cours d'utilisation. La prise d'un mutex n'est accordée que si (1) il est libre, et (2) la priorité de la tâche le demandant est strictement supérieure au plafond système, ou le plafond système est dû à la tâche.

Le protocole à priorité plafond assure :

- L'absence d'interblocage (appliquer le PCP à l'exemple des philosophes) ;
- Une tâche accédant à plusieurs mutex ne peut être bloquée au plus que par une seule section critique de priorité inférieure (voir Figure 4).

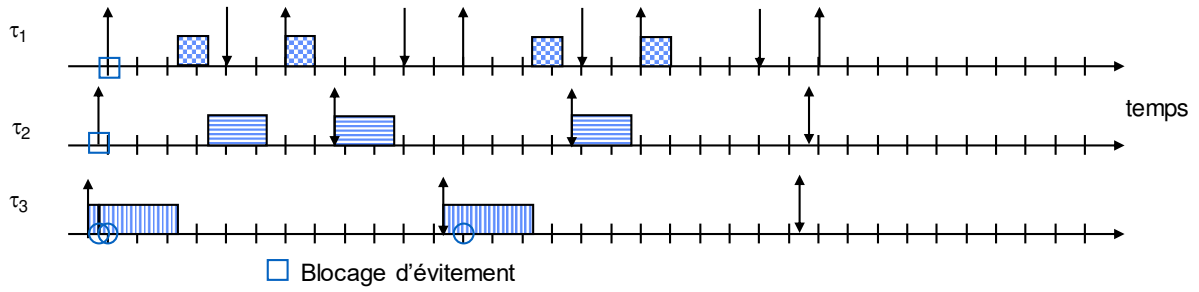


Figure 4 : Le protocole à priorité plafond garantit qu'au plus une section critique de priorité inférieure retarde une tâche

Dans la réalité, les RTOS proposant le protocole à priorité plafond en font une implémentation ayant les mêmes propriétés pire cas, mais beaucoup plus simple : le protocole à priorité plafond immédiat. Une tâche obtenant le mutex hérite directement de la priorité plafond du mutex. Sur l'exemple de la Figure 4, le comportement est le même en apparence (voir Figure 5), bien que l'héritage de priorité ait lieu en début de chaque section critique : dans ce cas, la tâche entrant en section critique hérite directement du plafond des ressources, soit la priorité de τ_1. Cela n'est cependant pas toujours le cas, ainsi, si une tâche indépendante de priorité intermédiaire entre celle de τ_1 et celle de τ_3 s'activait entre l'activation de τ_3 et l'activation de τ_1, celle-ci s'exécuterait avec PCP, alors qu'elle ne s'exécuterait pas en Immediate-PCP. Noter que ce protocole avait été proposé dans (Kaiser, 1982), mais que cet article français, paru 8 ans avant (Sha, Rajkumar, & Lehoczky, 1990), n'est malheureusement quasiment jamais cité.

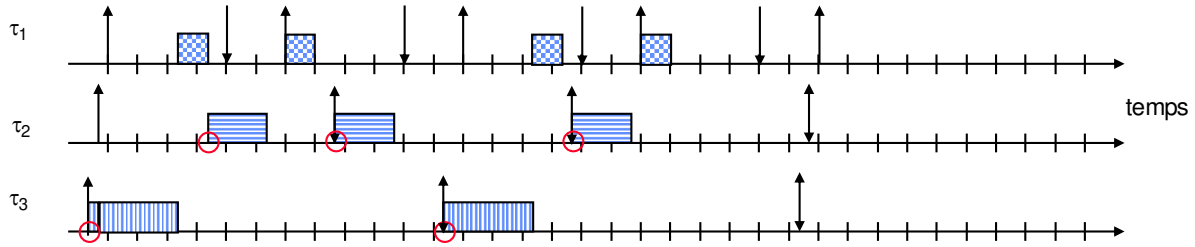


Figure 5 : Version immédiate du protocole à priorité plafond

Attention, dans ces trois protocoles, même une tâche n'utilisant pas de ressource peut se voir bloquer par une section critique de priorité inférieure, si celle-ci accède à un mutex dont le plafond est supérieur ou égal à la priorité de la tâche considérée, à cause de l'héritage de priorité.

Rappelons que la simulation n'est plus C-viable, T-viable ou r-viable avec des ressources critiques, et qu'il n'existe pas d'affectation de priorité optimale. Comment valider un tel système ?

En faisant une RTA modifiée : il suffit de considérer un pire cas qui consiste à supposer que, lors de la période d'activité initiée par l'instant critique, la (dans le cas PCP) ou les (dans le cas PIP) sections critiques les plus longues de tâches moins prioritaires que la tâche étudiée viennent juste de commencer. On suppose donc qu'il faudra que la tâche étudiée attende la pire durée de blocage due aux sections critiques de priorité inférieure démarrées « juste au pire moment ». Notons que la RTA ainsi obtenue devient une condition suffisante (mais non nécessaire) d'ordonnançabilité, car il est possible que le pire cas ne puisse jamais survenir, notamment lorsque les tâches sont périodiques. Nous ne cherchons pas à avoir une méthode exacte, car dans le cas de ressources, nous savons que le problème d'ordonnançabilité est NP-difficile au sens fort (Mok, 1983). Cette pire durée de blocage de la tâche τ_i par des sections critiques de priorité inférieure se note B_i. Dans la RTA, il suffit alors d'ajouter ce terme :

$$w_i^0(k) = B_i + kC_i$$

$$w_i^*(k) = B_i + kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)}{T_j} \right\rceil C_j$$

Comment calculer ce terme B_i ? C'est assez simple, voyons cela sur un exemple : nous considérons une table (voir Figure 6) dans laquelle nous plaçons la tâche qui nous intéresse sur une ligne, les tâches plus prioritaires ou ex-aequo au-dessus, les tâches moins prioritaires au-dessous. Nous plaçons une ressource (de façon équivalente un mutex) sur chaque colonne, et plaçons une durée de section critique ligne i colonne j si la tâche i possède une section critique de cette durée utilisant la ressource j. Ainsi sur l'exemple, la tâche 1 utilise la ressource 2 pendant 2 unités de temps.

Traçons un trait sous la ligne de la tâche étudiée, ici la tâche 2. Toute section critique possédant une valeur au-dessus et au-dessous du trait est susceptible de voir une section critique de priorité inférieure retarder la tâche étudiée. Par exemple, ici, la section critique de la tâche 4 sur la ressource 1 peut hériter d'une priorité supérieure ou égale à la priorité de la tâche 2, et ainsi retarder celle-ci.

Dans le cas PIP, chaque section critique concernée peut retarder la tâche une fois (elle peut avoir débuté juste avant l'instant critique), donc $B_{2,PIP}$=max(3)+max(1)+max(2,4)=8. Dans le cas PCP ainsi qu'Immediate-PCP, à cause du principe du plafond, au plus une seule section critique apparaissant au-dessus et au-dessous du trait a pu commencer, par conséquent $B_{2,PCP}$=max(3,1,2,4)=4.

	R_1	R_2	R_3	R_4	R_5	R_6
τ_1		2				5
τ_2	2					
τ_3		1	2			
τ_4	3			2		2
τ_5			4	3		
τ_6						
τ_7			6			4

Niveau de priorité

Figure 6 : Calcul du facteur de blocage B_2

Exercice 1

Calculer pour chaque tâche de la Figure 6 le facteur de blocage PCP, et le facteur de blocage PIP de chaque tâche.

Exercice 2

Proposer une adaptation du test hyperbolique et du test de L&L aux tâches partageant des ressources. Justifier le fait que le test ainsi adapté reste une condition suffisante d'ordonnançabilité.

Exercice 3

Soit le système de tâches sporadiques à échéance sur requête exprimé en AADL sur la Figure 7. La communication par data port est implémentée par un tableau noir, protégé par un mutex. Nous supposons que les sections critiques d'accès à ce tableau noir utilisent, aussi bien en lecture qu'en écriture, 0.5 unité de temps. L'implémentation est en POSIX, et un protocole de gestion de ressources a été choisi afin d'éviter l'inversion de priorité.

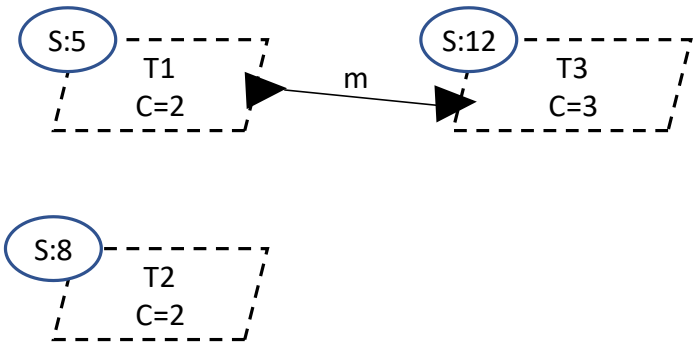


Figure 7 : Système à valider

a) Quel est le protocole de gestion de ressources choisi ?
b) Existe-t-il une affectation de priorités optimale dans ce contexte ? Si oui, la choisir, sinon, en choisir une qui a de bonnes chances de fonctionner.
c) Calculer pour chaque tâche son facteur de blocage.

d) Calculer le pire temps de réponse de chaque tâche, et conclure sur l'ordonnançabilité du système.
e) Représenter par une simulation le comportement du système en supposant que les tâches sont réveillées périodiquement, et que T3 se éveille initialement à l'instant 0, T2 à l'instant 0.1 et T1 à l'instant 0.2.

Contraintes de précédence

Lorsqu'il existe des **contraintes de précédence**, il nous faut intégrer un nouveau paramètre caractérisant le retard potentiel que peut prendre une tâche par rapport à sa « période escomptée ». En effet, considérons la Figure 8, sur laquelle T1 communique avec T2 via event data port. Globalement, la période de T2 est, comme T1, de 10 unités de temps. Cependant, entre 2 activations successives de T2, il peut y avoir moins de 10 unités de temps : si lors d'une exécution, en supposant que le message est envoyé par T1 en fin d'exécution, le temps de réponse de T1 vaut 3 unités de temps, mais que dans sa prochaine exécution le temps de réponse de T1 vaut presque 0 unités de temps, alors entre la première et la seconde activation de T2, seules 7 unités de temps se sont écoulées.

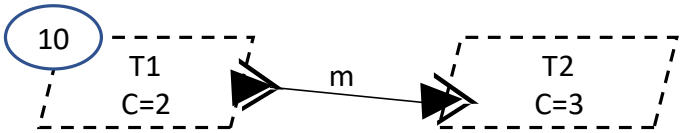


Figure 8 : Communication synchrone impliquant une contrainte de précédence T1 précède T2

Globalement la période de T2 est bien de 10 unités de temps, cependant elle peut être amenée à subir un retard dans sa période qui est égal au pire temps de réponse de T1. Ce retard est appelé gigue d'activation (en anglais *jitter*), et est dénoté J_i. Ici J_1=0, et J_2=TR(τ_1).

Théorème de l'instant critique avec gigue (Audsley, 1991) : la plus longue période d'activité pour des tâches avec gigue survient lorsque toutes les tâches sont activées simultanément après avoir subi leur pire gigue, puis exécutées périodiquement sans gigue.

Nous pouvons donc adapter la formule de calcul RTA au cas des gigues, tout simplement en considérant un instant critique à l'instant 0, mais que toutes les tâches τ_i considérées dans le calcul de la période d'activité sont activées à la date $-J_i$. Cela revient, dans la RBF, lorsque $J_i<T_i$, à avoir la première « marche » à l'instant 0 (comme sans gigue), mais la seconde à la date T_i-J_i, puis les autres espacées de T_i unités de temps. Ainsi, sur la Figure 9, on voit que pour J_1=1, le second réveil de τ_1 se produit à la date 3, puis les autres périodiquement de période 4, aux dates 7, 11, 15, etc. Cependant, pour J_1=3, le second réveil de τ_1 se produit à la date 1, puis les autres aux dates 5, 9, 13, etc., ce qui augmente grandement la durée de la période d'activité.

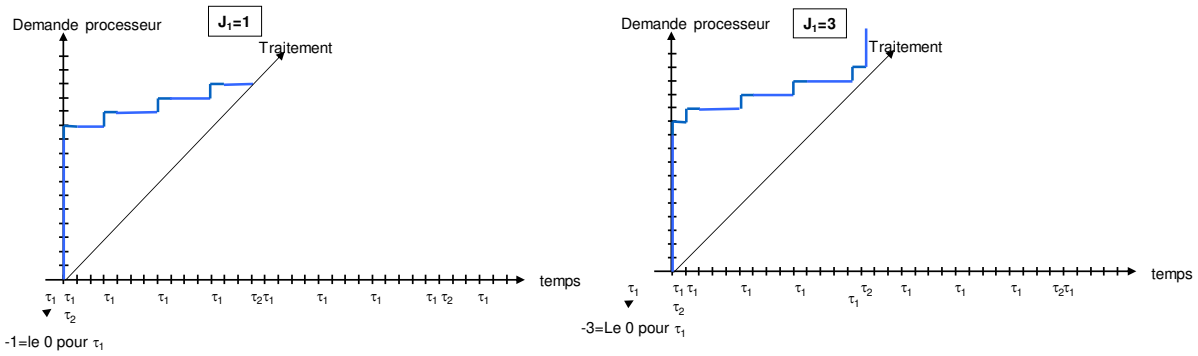


Figure 9 : RBF comparées de {τ_1 <C_1=1, J_1=1 ou 3, T_1=4>, τ_2 <C_2=10, J_2=0, T_2=14>}

Graphiquement, on peut voir que la RTA est J-viable, i.e., le temps de réponse ne peut pas se voir rallonger en réduisant la gigue.

La formule de calcul RTA devient donc :

$k = 0$
Répéter
$\quad k = k + 1$
$\quad w_i^0(k) = B_i + kC_i$
$\quad$ Trouver point fixe non nul $w_i^*(k) = B_i + kC_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^*(k)+J_j}{T_j} \right\rceil C_j$
$\quad TR_i(k) = w_i^*(k) - (k-1)T_i + J_i$
Jusqu'à $TR_i(k) \leq T_i$
$TR_i = \max_{j=1..k}\left(TR_i(j)\right)$

Nous voyons que la gigue apparaît à deux endroits du calcul : dans la recherche du point fixe, le nombre de réveils d'une tâche τ_j ayant une gigue est non plus calculé sur [0..t[mais sur [-J_j..t[d'où l'ajout de J_j dans le numérateur de la partie entière supérieure.

De plus, la tâche τ_i étudiée, comme les autres, est supposée être réveillée non pas à l'instant 0, mais à l'instant -J_i puisqu'elle subit sa gigue maximale. Par conséquent, on ajout J_i au temps de réponse, puisque $w_i^*(k)$ nous donne la date de fin de période d'activité, mais que la date d'activation de la tâche étudiée n'est pas 0 mais -J_i.

Exercice 4

Soit le système de tâches à échéance sur requête exprimé en AADL sur la Figure 10. La communication par data port est implémentée par un tableau noir, protégé par un mutex. Nous supposons que les sections critiques d'accès à ce tableau noir utilisent, aussi bien en lecture qu'en écriture, 0.5 unité de temps. L'implémentation est en VxWorks, et un protocole de gestion de ressources a été choisi afin d'éviter l'inversion de priorité. La communication par message utilise des messageQueues, gérant le buffer de message en exclusion mutuelle avec le même protocole de gestion de ressource, et nous supposons qu'envoyer ou lire un message existant prend au pire 0,6 unités de temps en exclusion mutuelle.

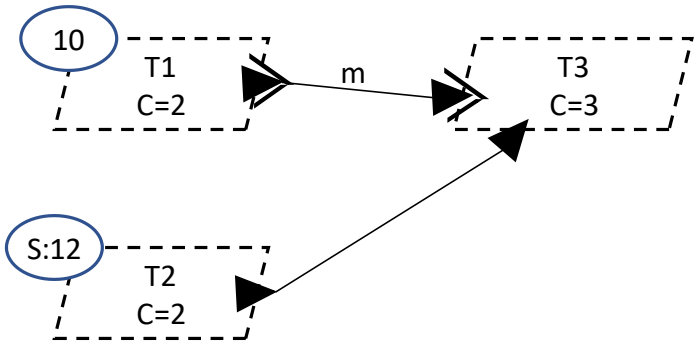


Figure 10 : Système avec exclusion mutuelle et précédence à valider

a) Quel est le protocole de gestion de ressources choisi ?
b) Existe-t-il une affectation de priorités optimale dans ce contexte ?

c) Si nous choisissions une priorité de T3 supérieure à celle de T1, que se passerait-il pour le calcul de la gigue J_3 ? En déduire une affectation de priorités qui nous permettra d'éviter cela.
d) Calculer pour chaque tâche son facteur de blocage.
e) Calculer le pire temps de réponse de chaque tâche, et conclure sur l'ordonnançabilité du système.

Bibliographie

Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times.* Computer Science Report YCS 164, York.

Kaiser, C. (1982). Exclusion mutuelle et ordonnancement par priorité. *Technique et science informatiques 1 (1)*, 59-68.

Mok, A. K.-L. (1983). *Fundamental design problems of distributed systems for the hard-real-time environment.* thesis, Massachusetts Institute of Technology.

Sha, L., Rajkumar, R., & Lehoczky, J. P. (1990). Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on computers 39 (9)*, 1175-1185.

Validation temporelle par Model Checking

Emmanuel GROLLEAU

Motivations

- Quel que soit le contexte industriel, l'informatique critique répond à des standards de sûreté de fonctionnement
  - Avionique DO-178B/C, DO-254: Design Assurance Level DAL A/B/C/D/E
  - Automobile ISO 26262: Automotive Design Insurance Level ASIL 4/3/2/1/QM
  - Ferroviaire/autre: CENELEC 20126/128/128: Software Insurance Level SIL 4/3/2/1/(0)
  - Drone: Joint Authorities for Rulemaking of Unmanned Systems (JARUS) définit AMC 1309 – en cours d'élaboration

Sûreté de fonctionnement

- Des méthodes sont mises en place pour garantir au mieux la sûreté
- On peut faire des simulations, puis des centaines ou milliers d'heures de tests => coût élevé et fiabilité basse
- Trois grandes familles de méthodes de vérification formelle
  - **Vérification par modèle** (*model checking – RdP, automate temporisé, TLA+, etc.*)
  - Preuve assistée (*theorem proving – Coq, Isabelle, etc.*)
  - Raffinement de spécification (méthode B)

Carl Adam Petri, 1962

RÉSEAUX DE PETRI

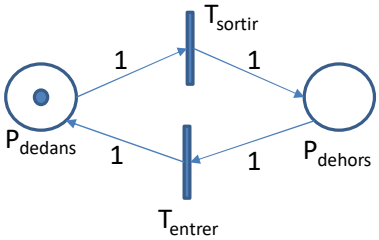
Modèle

- Un RdP est un modèle
  - Modèle = abstraction représentative de ce qui est modélisé
  - En informatique, et V&V, permet des analyses et preuves permettant d'augmenter la confiance dans un système
  - Généralement notation mathématique (ou graphique pour lequel une équivalence mathématique existe)
- Un RdP classique ne représente pas ce qui sera, mais ce qui peut être
- De base, il n'a pas un comportement temporel ou événementiel déterministe. Lorsqu'une transition est franchissable, elle n'est pas forcée, elle peut être franchie (on ne dit pas quand), ou ne pas l'être

Expressivité d'un modèle

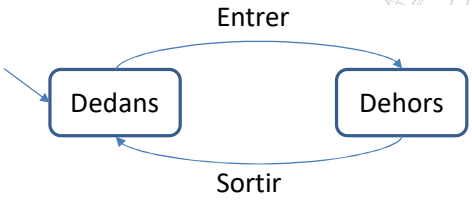
- Machine de Turing : puissance d'expression d'un programme exécuté sur architecture von Neumann
- Réseau de Petri : sait compter, mais ne sait pas représenter un test à 0 pour une place qui compte
- Automate fini : ne sait représenter que les langages réguliers (représentables par expression régulière), ne sait pas compter, ne sait pas représenter $a^n b^n$
- Plus de puissance = plus de complexité d'analyse

Représentation graphique d'un RdP



RdP

Equivalence



Automate fini

jeton

place

transition

n → **arc** valué

[1,0] : **marquage initial**

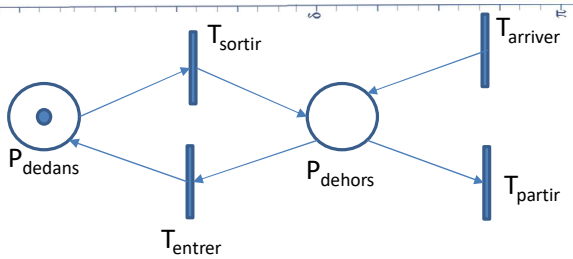
T_{sortir} est **franchissable** (ou **tirable**)

le **franchissement** de T_{sortir} consomme un jeton dans P_{dedans} et produit un jeton dans P_{dehors}. Après franchissement le marquage est [0,1].

Un RdP est un graphe **bipartite**

Ce RdP est **sauf**, par construction, chaque place ne peut contenir qu'un jeton au plus.

Quelques exemples simples



$T_{arriver}$ est une transition source (pas de précondition)

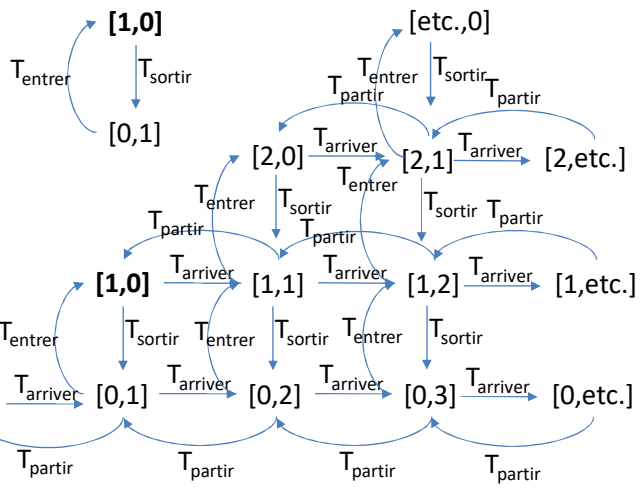
T_{partir} est une transition puits (pas de postcondition)

Ce RdP est **non borné**. Potentiellement certaines places (ici toutes) peuvent contenir une infinité de jetons

Le marquage M=[0,0] est-il atteignable ? Si oui, exhiber une séquence de franchissements de transitions y menant.

Graphe des marquages

- Dans le cas borné, le GM est borné



- Dans le cas non borné, il ne l'est pas

Formalisation

- Réseau de Petri : N=(P,T,W) où
  - P ensemble fini de places
  - T ensemble fini de transitions $(P \cap T)=\varnothing$
  - W: $P \times T \cup T \times P \rightarrow \mathbb{N}$ fonction de valuation
    - ✓Pré $W_{|P \times T}$ fonction de précondition
    - ✓Post $W_{|T \times P}$ fonction de postcondition
- Etat d'un RdP : fonction de marquage $M : P \rightarrow \mathbb{N}$, ou vecteur de marquage $M \in \mathbb{N}^{|P|}$
- Marquage initial : $M_0 : P \rightarrow \mathbb{N}$

Comportement

- $t \in T$ est franchissable à partir du marquage M, noté **M(t>**
  - Ssi $\forall p \in P$, $M(p) \geq W(p,t)$
- $t \in T$ est franchie à partir du marquage M, pour donner M', noté **M(t>M'**
  - Si M(t>
  - $\forall p \in P$, $M'(p)=M(p)-W(p,t)+W(t,p)$
- La séquence $\sigma=t_1t_2...t_n$ est franchie, noté $M(\sigma>M_n$
  - Ssi $\exists M_1,...,M_n$ tels que $M(t_1>M_1(t_2>M_2(...M_{n-1}(t_n>M_n$

Langage d'un RdP étiqueté

- Si on étiquette les transitions d'un RdP sur un alphabet Σ, on mot produit par le RdP est la suite des étiquettes des transitions d'une séquence franchissable $M(\sigma>M_n$
- Pour simplifier les notations, si on considère que les transitions sont nommées sur des lettres de Σ
- Langage du RdP: $L(N,M_0)=\{\sigma \in T^* \mid M_0(\sigma>\}$
- Langage terminal: $L(N,M_0,\mathbb{M})=\{\sigma \in T^* \mid \exists M \in \mathbb{M}, M_0(\sigma>M\}$
- Marquages accessibles du RdP :
  - $Acc(N,M_0)=\{M \mid \exists\ \sigma \in L(N,M_0),\ M_0(\sigma>M\}$

Expressivité

- Proposer un automate fini dont le langage est $a^n b^n$ pour $n \leq 2$
- Proposer un RdP pour ce langage
- Proposer un automate fini dont le langage est $a^n b^n$ pour n quelconque
- Proposer un RdP pour ce langage

Etude d'un RdP

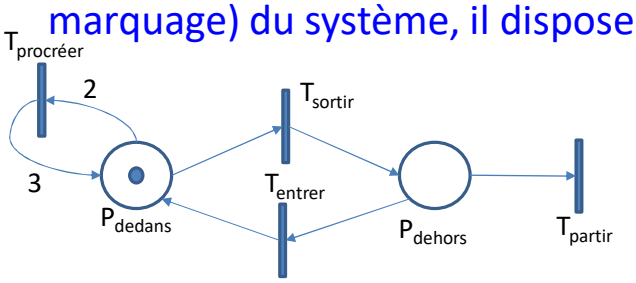
- Propriétés de vivacité
  - Pseudo-vivant : aucun état de blocage
  - Vivant : tout état (et donc tout état souhaitable) est toujours atteignable
- Sûreté
  - Rien de mauvais ne pourra se produire
    - ✓ Place observateur, contenant des jetons lorsque quelque chose à éviter se produit

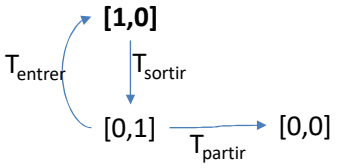
Formalisation propriétés

- (N,M_0) admet une **séquence infinie**
  - $\Leftrightarrow \exists \sigma_\infty \in T^\infty / \forall \sigma$ préfixe fini de σ_∞, $M(\sigma>$
- (N,M_0) est **pseudo-vivant**
  - $\Leftrightarrow \forall M \in Acc(N,M_0), \exists t \in T, M(t>$
- (N,M_0) est **quasi-vivant**
  - $\Leftrightarrow \forall t \in T, \exists M \in Acc(N,M_0)/M(t>$
  - $\Leftrightarrow \forall t \in T, \exists \sigma \in L(N,M_0)/M_0(\sigma t>$
- (N,M_0) est **vivant**
  - $\Leftrightarrow \forall M \in Acc(N,M_0)$, (N,M) quasi-vivant
  - $\Rightarrow (N,M_0)$ quasi-vivant et pseudo-vivant
  - $\Leftarrow (N,M_0)$ quasi-vivant et M_0 état d'accueil

Vivacité

- Dans les deux GM précédents, le RdP est **vivant**. Quel que soit l'état (le marquage) du système, il dispose encore de toutes ses fonctionnalités.



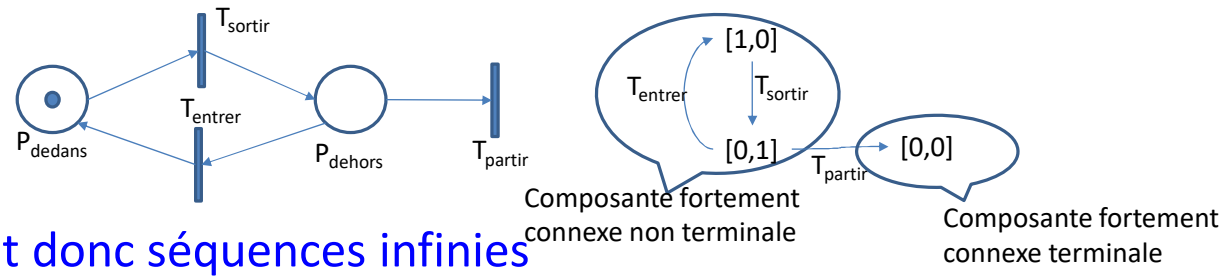


- Ici, il existe un **blocage**, le marquage [0,0] est un marquage **mort** (état puits).
- A partir du marquage [1,0] le réseau n'est pas **quasi-vivant**, car $T_{procréer}$ n'est jamais franchissable
- A partir du marquage [2,0] le réseau est **quasi-vivant**, mais pas **pseudo-vivant**

Propriétés relatives à l'état

- M **état d'accueil** de (N,M_0)
  - $\Leftrightarrow \forall M' \in Acc(N,M_0), \exists \sigma \in T^*/M'(\sigma>M$
- (N,M_0) est **borné**
  - $\Leftrightarrow Acc(N,M_0)$ est fini
  - $\Leftrightarrow \forall p \in P, \exists k_p \in \mathbb{N}/M(p) \leq k_p$
  - $\Leftrightarrow \exists k \in \mathbb{N}/\forall p \in P, M(p) \leq k$
- Monotonie: $M(t>M' \wedge M_1 \geq M \Rightarrow M_1(t>M'_1 \wedge M'_1 \geq M'$
- $M(\sigma>M' \wedge M' \geq M \Rightarrow M(\sigma^n>$
  - Si $M'>M$ réseau non borné

Analyse par graphe des marquages

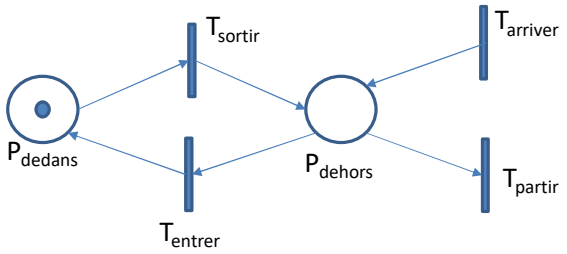


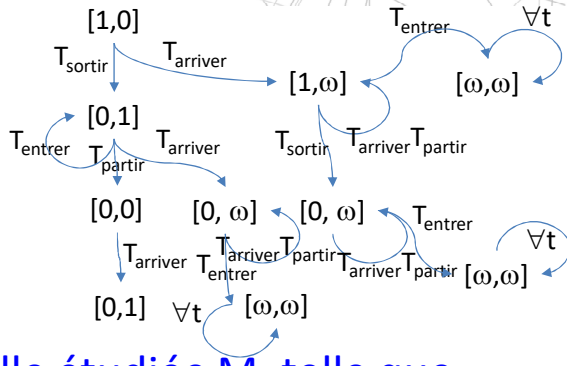
- 1 circuit donc séquences infinies
- Sommet sans successeur donc non pseudo-vivant (donc non vivant, pas d'état d'accueil)
- Toutes les transitions apparaissent donc quasi-vivant
- Borné par 1

Analyse par graphe des marquages

- Cas (N,M_0) borné
  - (N,M_0) admet une séquence infinie ⇔ GM possède un circuit
  - (N,M_0) vivant ⇔ ∀composante fortement connexe terminale contient un arc étiqueté par toute transition de T
  - (N,M_0) état d'accueil ⇔ ∃ une unique composante fortement connexe (terminale)
- (N,M_0) quelconque
  - (N,M_0) pseudo-vivant ⇔ tout sommet admet un successeur
  - (N,M_0) quasi-vivant ⇔ le GM contient un arc étiqueté par toute transition de T

Arbre de couverture

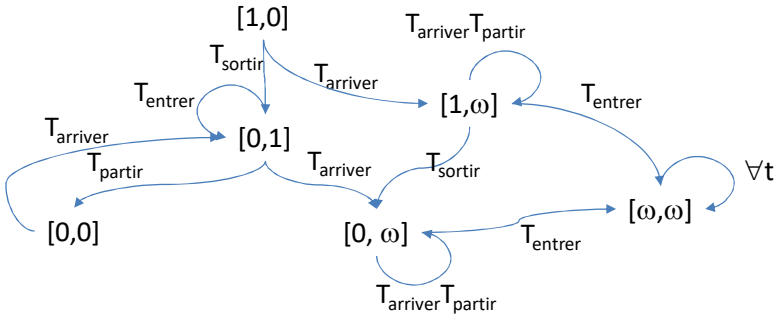




- En largeur ou profondeur, à partir de la feuille étudiée M, telle que $M_0(\sigma>M$. Si M existe dans son préfixe (i.e. $\sigma=\sigma_1\sigma_2$ / $M_0(\sigma_1>M(\sigma_2>M)$, ne pas le traiter. Sinon
  - $\forall t/M(t>M'$, si il existe sur le chemin $M_0(\sigma>M'$ un marquage $M/M'>M$, remplacer chaque composante supérieure par ω.
  - Règles algébriques: $\forall n\in\mathbb{N}$, $\omega+n=\omega-n=\omega$, $\omega>n$, $\omega\leq\omega$

Graphe de couverture

- Fusionner les états identiques



Complexité analyse

- Borné par k : PSPACE-complet (les plus compliqués des problèmes requérant une taille polynomiale pour être résolus, suspecté d'être plus complexe que NP-complet)
- Accessibilité
  - PSPACE-complet si les transitions du RdP ont le même nombre de places en pré qu'en post
  - PSPACE-complet pour les RdP saufs (bornés par 1)
  - NP-complet si pas de cycle
  - Double exponentiel si au plus 5 places
  - EXPSPACE-complet pour les RdP symétriques (∀transition, il existe une transition « l'annulant »)
  - Etc.

Complexité analyse

- Vivacité
  - PSPACE-complet pour RdP saufs
- Sans blocage
  - PSPACE-complet pour RdP saufs s'ils sont à chemin simple (∃ une seule séquence infinie de transitions)
- Etat d'accueil : décidable

Modélisation par RdP

- Proposer un modèle RdP pour le programme parallèle suivant

<u>Faire toujours</u>
Attendre 2 vis
Assembler et créer élément

<u>Faire toujours</u>
Produire 1 vis

- Proposer un modèle RdP pour le programme parallèle suivant

<u>Faire toujours</u>
Attendre 2 vis et 2 boulons
Assembler et créer élément

<u>Faire toujours</u>
Produire 1 vis

<u>Faire toujours</u>
Produire 3 boulons

- Proposer un modèle RdP pour le programme suivant

Si i=0 alors A
Sinon B

Valeur i